Authentication and Authorization

Authentication and authorization form the foundation of application security. This section creates comprehensive specifications for user authentication, JWT token management, role-based access control, and secure session handling.

Exercise 1.1: JWT Authentication System

Create comprehensive JWT authentication specifications:

~~~
Design detailed JWT authentication system for our chatbot application:

**Use the Web Search MCP server to research current JWT security best practices, then create:**

**JWT Token Structure and Configuration**:
```typescript
interface JWTConfiguration {
  // Token Structure
  tokenStructure: {
    header: {
      alg: "RS256"; // RSA SHA-256 for asymmetric signing
      typ: "JWT";
      kid: "Key ID for key rotation";
    };

    payload: {
      // Standard Claims
      iss: "chatbot-api.com"; // Issuer
      sub: "user-uuid"; // Subject (user ID)
      aud: "chatbot-client"; // Audience
      exp: "Expiration timestamp";
      iat: "Issued at timestamp";
      nbf: "Not before timestamp";
      jti: "JWT ID for revocation";

      // Custom Claims
      email: "user@example.com";
      role: "user | admin | premium";
      permissions: ["chat:read", "chat:write", "profile:update"];
      sessionId: "session-uuid";
      mfa: "boolean - MFA completed";
    };
  };

  // Token Configuration
  tokenConfiguration: {
    accessToken: {
      algorithm: "RS256";
      expiresIn: "15 minutes";
      issuer: "chatbot-api.com";
      audience: "chatbot-client";
    };

    refreshToken: {
      algorithm: "RS256";
      expiresIn: "7 days";
      storage: "Secure HTTP-only cookie";
      rotation: "Rotate on each use";
    };

    keyManagement: {
      keyRotation: "Monthly automatic rotation";
      keyStorage: "AWS Secrets Manager";
      keyBackup: "Cross-region key backup";
      algorithm: "RSA 2048-bit keys";
    };
  };

  // Security Controls
  securityControls: {
    tokenValidation: {
      signatureVerification: "Verify RSA signature";
      expirationCheck: "Strict expiration validation";
      audienceValidation: "Validate intended audience";
      issuerValidation: "Validate trusted issuer";
      clockSkew: "5 minutes maximum skew";
    };

    tokenRevocation: {
      blacklist: "Redis-based token blacklist";
      jtiTracking: "Track JWT IDs for revocation";
      sessionInvalidation: "Invalidate all user sessions";
      gracePeriod: "5 minute grace period for clock skew";
    };
  };
}
```

**Authentication Flow Implementation**:
```typescript
interface AuthenticationFlow {
  // Registration Flow
  registrationFlow: {
    steps: [
      "Email validation and uniqueness check",
      "Password strength validation",
      "Account creation with pending status",
      "Email verification token generation",
      "Verification email sending",
      "Email confirmation and account activation"
    ];

    security: {
      emailVerification: "Required before account activation";
      passwordHashing: "bcrypt with 12 rounds";
      rateLimiting: "5 registration attempts per IP per hour";
      captcha: "reCAPTCHA for suspicious activity";
    };
  };

  // Login Flow
  loginFlow: {
    steps: [
      "Email and password validation",
      "Account status verification",
      "Password verification with timing attack protection",
      "MFA challenge (if enabled)",
      "Session creation and token generation",
      "Secure token delivery"
    ];

    security: {
      bruteForceProtection: "Account lockout after 5 failed attempts";
      timingAttackPrevention: "Constant-time password comparison";
      rateLimiting: "10 login attempts per IP per 15 minutes";
      deviceFingerprinting: "Track device characteristics";
    };
  };

  // Token Refresh Flow
  tokenRefreshFlow: {
    steps: [
      "Refresh token validation",
      "Session validity verification",
      "New access token generation",
      "Refresh token rotation",
      "Secure token delivery"
    ];

    security: {
      tokenRotation: "Rotate refresh token on each use";
      familyTracking: "Track token families for security";
      reuseDetection: "Detect refresh token reuse attacks";
      sessionBinding: "Bind tokens to specific sessions";
    };
  };
}
```

Research current JWT security vulnerabilities and mitigation strategies using the Web Search MCP server.
~~~

Expected Outcome

- Comprehensive JWT authentication system with proper security controls
- Secure token structure with appropriate claims and validation
- Multi-step authentication flows with security measures
- Token rotation and revocation mechanisms
- Protection against common JWT vulnerabilities

Exercise 1.2: Multi-Factor Authentication

Implement comprehensive multi-factor authentication specifications:

~~~
Create detailed MFA implementation for enhanced security:

**MFA Methods and Configuration**:
```typescript
interface MFAImplementation {
  // Supported MFA Methods
  mfaMethods: {
    totp: {
      name: "Time-based One-Time Password";
      implementation: "RFC 6238 TOTP algorithm";
      secretLength: "32 bytes (256 bits)";
      timeStep: "30 seconds";
      windowSize: "1 (allow ±30 seconds)";
      apps: ["Google Authenticator", "Authy", "1Password"];
    };

    sms: {
      name: "SMS-based verification";
      implementation: "6-digit numeric codes";
      expiration: "5 minutes";
      rateLimiting: "3 SMS per 15 minutes";
      provider: "AWS SNS with fallback";
    };

    email: {
      name: "Email-based verification";
      implementation: "6-digit alphanumeric codes";
      expiration: "10 minutes";
      rateLimiting: "5 emails per hour";
      template: "Branded email template";
    };

    backupCodes: {
      name: "Single-use backup codes";
      implementation: "8-character alphanumeric codes";
      quantity: "10 codes generated";
      usage: "Single use, mark as consumed";
      regeneration: "User-initiated regeneration";
    };
  };

  // MFA Enrollment Process
  enrollmentProcess: {
    totpEnrollment: {
      steps: [
        "Generate secret key",
        "Display QR code and manual entry option",
        "User scans QR code in authenticator app",
        "User enters verification code",
        "Verify code and activate TOTP",
        "Generate and display backup codes"
      ];

      security: {
        secretGeneration: "Cryptographically secure random generation";
        qrCodeSecurity: "Time-limited QR code display";
        verificationRequired: "Must verify before activation";
        backupCodesRequired: "Generate backup codes during enrollment";
      };
    };

    smsEnrollment: {
      steps: [
        "Phone number validation and formatting",
        "Send verification SMS",
        "User enters verification code",
        "Verify code and activate SMS MFA"
      ];

      security: {
        phoneValidation: "E.164 format validation";
        rateLimiting: "Prevent SMS flooding";
        carrierValidation: "Validate against known carriers";
        voipDetection: "Detect and warn about VoIP numbers";
      };
    };
  };

  // MFA Challenge Process
  challengeProcess: {
    challengeGeneration: {
      timing: "After successful password authentication";
      methods: "Present available MFA methods";
      fallback: "Allow fallback to backup codes";
      timeout: "5 minutes to complete challenge";
    };

    codeValidation: {
      totp: "Validate against current and previous time windows";
      sms: "Validate against sent code with expiration";
      email: "Validate against sent code with expiration";
      backupCode: "Validate and mark as consumed";
    };

    failureHandling: {
      maxAttempts: "3 attempts per challenge";
      lockoutDuration: "15 minutes after max attempts";
      alerting: "Security alert for repeated failures";
      logging: "Comprehensive audit logging";
    };
  };
}
```

**MFA Security Controls**:
```typescript
interface MFASecurityControls {
  // Security Measures
  securityMeasures: {
    secretProtection: {
      storage: "Encrypted in database with user-specific keys";
      transmission: "Never transmit secrets in plain text";
      display: "Mask secrets in UI, show only during setup";
      rotation: "Allow secret regeneration";
    };

    codeGeneration: {
      randomness: "Cryptographically secure random number generation";
      uniqueness: "Ensure codes are unique within time window";
      expiration: "Strict expiration enforcement";
      replayPrevention: "Prevent code reuse";
    };

    rateLimiting: {
      perUser: "Limit MFA attempts per user";
      perIP: "Limit MFA attempts per IP address";
      perMethod: "Method-specific rate limits";
      globalLimits: "System-wide rate limiting";
    };
  };

  // Recovery Mechanisms
  recoveryMechanisms: {
    backupCodes: {
      generation: "Generate during MFA setup";
      storage: "Hashed storage, plain text display once";
      usage: "Single-use codes";
      regeneration: "User-initiated with authentication";
    };

    accountRecovery: {
      process: "Multi-step identity verification";
      documentation: "Government ID verification";
      timeline: "24-48 hour verification process";
      notification: "Email notification of recovery attempts";
    };

    adminOverride: {
      authorization: "Require multiple admin approvals";
      logging: "Comprehensive audit trail";
      notification: "Notify user of admin override";
      temporaryAccess: "Limited time access for recovery";
    };
  };
}
```

Research current MFA best practices and security considerations using the Web Search MCP server.
~~~

Expected Outcome

- Comprehensive MFA system with multiple authentication methods
- Secure enrollment and challenge processes
- Proper secret management and code generation
- Recovery mechanisms for account access
- Rate limiting and security controls

Exercise 1.3: Role-Based Access Control (RBAC)

Design comprehensive RBAC system specifications:

~~~
Create detailed RBAC implementation for authorization:

**RBAC Model and Structure**:
```typescript
interface RBACSystem {
  // Role Definitions
  roleDefinitions: {
    user: {
      name: "Standard User";
      description: "Regular chatbot user with basic permissions";
      permissions: [
        "chat:read",
        "chat:write",
        "conversation:create",
        "conversation:read",
        "conversation:update",
        "conversation:delete",
        "profile:read",
        "profile:update"
      ];
      restrictions: [
        "Cannot access admin functions",
        "Limited to own conversations",
        "Rate limited API access"
      ];
    };

    premium: {
      name: "Premium User";
      description: "Premium subscriber with enhanced features";
      permissions: [
        "All user permissions",
        "chat:priority",
        "conversation:export",
        "analytics:read",
        "support:priority"
      ];
      restrictions: [
        "Higher rate limits",
        "Access to premium AI models",
        "Priority support queue"
      ];
    };

    moderator: {
      name: "Content Moderator";
      description: "Moderates content and user interactions";
      permissions: [
        "All user permissions",
        "moderation:read",
        "moderation:write",
        "user:read",
        "conversation:moderate",
        "report:create"
      ];
      restrictions: [
        "Cannot modify user accounts",
        "Limited to moderation functions"
      ];
    };

    admin: {
      name: "System Administrator";
      description: "Full system access for administration";
      permissions: [
        "All permissions",
        "user:create",
        "user:update",
        "user:delete",
        "system:configure",
        "audit:read",
        "metrics:read"
      ];
      restrictions: [
        "Requires MFA for sensitive operations",
        "All actions logged and audited"
      ];
    };
  };

  // Permission Structure
  permissionStructure: {
    format: "resource:action";
    resources: ["chat", "conversation", "user", "profile", "system", "audit"];
    actions: ["create", "read", "update", "delete", "moderate", "configure"];

    examples: {
      "chat:read": "Read chat messages";
      "conversation:create": "Create new conversations";
      "user:update": "Update user information";
      "system:configure": "Configure system settings";
    };
  };

  // Dynamic Permissions
  dynamicPermissions: {
    contextualPermissions: {
      "conversation:read": "Only if user owns conversation or has moderation role";
      "profile:update": "Only for own profile unless admin";
      "user:delete": "Cannot delete own account, admin only";
    };

    conditionalPermissions: {
      timeBasedAccess: "Admin functions only during business hours";
      locationBasedAccess: "Sensitive operations from approved locations";
      deviceBasedAccess: "Admin functions from registered devices";
    };
  };
}
```

**Authorization Implementation**:
```typescript
interface AuthorizationImplementation {
  // Permission Checking
  permissionChecking: {
    middleware: {
      implementation: "Express middleware for route protection";
      caching: "Cache user permissions for performance";
      fallback: "Deny access by default";
      logging: "Log all authorization decisions";
    };

    checkingLogic: {
      roleBasedCheck: "Verify user has required role";
      permissionBasedCheck: "Verify specific permission";
      contextualCheck: "Check resource ownership or context";
      hierarchicalCheck: "Check role hierarchy and inheritance";
    };
  };

  // Resource Protection
  resourceProtection: {
    apiEndpoints: {
      protection: "Protect all API endpoints with appropriate permissions";
      documentation: "Document required permissions in OpenAPI spec";
      testing: "Automated tests for authorization logic";
    };

    dataAccess: {
      rowLevelSecurity: "Filter data based on user permissions";
      fieldLevelSecurity: "Hide sensitive fields based on role";
      auditTrail: "Log all data access attempts";
    };
  };

  // Role Management
  roleManagement: {
    roleAssignment: {
      process: "Admin-only role assignment";
      approval: "Multi-admin approval for sensitive roles";
      notification: "Notify user of role changes";
      audit: "Complete audit trail of role changes";
    };

    roleInheritance: {
      hierarchy: "Admin > Moderator > Premium > User";
      inheritance: "Higher roles inherit lower role permissions";
      override: "Explicit permission overrides";
    };
  };
}
```

Research current RBAC implementation patterns and authorization best practices using the Web Search MCP server.
~~~

Expected Outcome

- Comprehensive RBAC system with well-defined roles and permissions
- Dynamic and contextual permission checking
- Proper resource protection at API and data levels
- Role management with audit trails and approval processes
- Hierarchical role inheritance and permission override capabilities

Exercise 1.4: Session Management and Security

Implement secure session management specifications:

~~~
Create detailed session management and security controls:

**Session Architecture**:
```typescript
interface SessionManagement {
  // Session Structure
  sessionStructure: {
    sessionData: {
      sessionId: "Cryptographically secure random UUID";
      userId: "User identifier";
      createdAt: "Session creation timestamp";
      lastAccessedAt: "Last activity timestamp";
      expiresAt: "Session expiration timestamp";
      ipAddress: "Client IP address";
      userAgent: "Client user agent string";
      deviceFingerprint: "Device identification hash";
      mfaCompleted: "MFA completion status";
      permissions: "Cached user permissions";
    };

    sessionStorage: {
      primary: "Redis for fast access";
      backup: "DynamoDB for persistence";
      encryption: "AES-256 encryption at rest";
      replication: "Cross-AZ replication for availability";
    };
  };

  // Session Lifecycle
  sessionLifecycle: {
    creation: {
      trigger: "Successful authentication";
      generation: "Secure random session ID";
      storage: "Store in Redis and DynamoDB";
      cookie: "Set secure HTTP-only cookie";
    };

    validation: {
      frequency: "On every authenticated request";
      checks: [
        "Session exists and not expired",
        "IP address consistency (with allowance for mobile)",
        "User agent consistency",
        "Device fingerprint validation",
        "User account status"
      ];
    };

    renewal: {
      trigger: "Activity within session";
      extension: "Extend expiration time";
      limits: "Maximum session duration limits";
      notification: "Warn user of impending expiration";
    };

    termination: {
      triggers: [
        "User logout",
        "Session expiration",
        "Security violation",
        "Admin termination",
        "Password change"
      ];
      cleanup: "Remove from all storage systems";
      notification: "Notify user of termination reason";
    };
  };

  // Security Controls
  sessionSecurity: {
    cookieConfiguration: {
      httpOnly: true;
      secure: true;
      sameSite: "Strict";
      domain: "Specific domain only";
      path: "Application path";
      maxAge: "Session duration";
    };

    securityValidation: {
      ipValidation: {
        strict: "Require exact IP match for admin sessions";
        flexible: "Allow IP changes for mobile users";
        geolocation: "Alert on significant location changes";
      };

      deviceValidation: {
        fingerprinting: "Browser and device characteristics";
        consistency: "Validate device consistency";
        newDevice: "Require additional verification for new devices";
      };

      concurrentSessions: {
        limit: "Maximum 5 concurrent sessions per user";
        policy: "Terminate oldest session when limit exceeded";
        notification: "Notify user of new session creation";
      };
    };
  };
}
```

**Session Monitoring and Anomaly Detection**:
```typescript
interface SessionMonitoring {
  // Anomaly Detection
  anomalyDetection: {
    suspiciousActivity: {
      rapidLocationChange: "Detect impossible travel scenarios";
      unusualAccessPatterns: "Detect abnormal usage patterns";
      deviceChanges: "Monitor frequent device changes";
      timePatterns: "Detect unusual access times";
    };

    securityAlerts: {
      concurrentLogins: "Multiple simultaneous logins from different locations";
      bruteForceAttempts: "Repeated failed authentication attempts";
      privilegeEscalation: "Attempts to access unauthorized resources";
      dataExfiltration: "Unusual data access patterns";
    };

    responseActions: {
      automaticTermination: "Terminate suspicious sessions";
      userNotification: "Alert user of suspicious activity";
      adminAlert: "Notify security team of threats";
      accountLockdown: "Temporary account suspension for severe threats";
    };
  };

  // Session Analytics
  sessionAnalytics: {
    metrics: {
      sessionDuration: "Average and distribution of session lengths";
      concurrentSessions: "Peak concurrent session counts";
      deviceTypes: "Distribution of device types and browsers";
      geographicDistribution: "User location analytics";
    };

    reporting: {
      securityDashboard: "Real-time security metrics";
      weeklyReports: "Session security summary reports";
      anomalyReports: "Detailed anomaly detection reports";
      complianceReports: "Session management compliance reports";
    };
  };
}
```

Set up Agent Hooks to automatically validate session security configurations and update security documentation when authentication code is modified.

Research current session management security best practices and attack prevention strategies using the Web Search MCP server.
~~~

Expected Outcome

- Comprehensive session management with secure lifecycle handling
- Advanced security controls with device and IP validation
- Anomaly detection and automated threat response
- Session monitoring and analytics for security insights
- Compliance with security best practices and standards

Key Deliverables

After completing the Authentication and Authorization exercises, you should have:

1. **JWT Authentication System**: Secure token-based authentication with proper validation
2. **Multi-Factor Authentication**: Comprehensive MFA with multiple methods and recovery
3. **Role-Based Access Control**: Granular permission system with dynamic authorization
4. **Session Management**: Secure session handling with anomaly detection
5. **Security Monitoring**: Comprehensive monitoring and threat detection capabilities

Integration with Advanced Features

- **Agent Steering**: For secure coding standards and authentication best practices
- **Agent Hooks**: For automated security validation and compliance checking

Next Steps

With comprehensive authentication and authorization established, you're ready to implement data protection strategies that ensure user privacy, regulatory compliance, and secure data handling throughout your chatbot application.

Previous Next

© 2008 - 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.